

Week 07: Parser combinators

Functional Programming in Lean

Learning goals

- ▶ Parser result and remaining input
- ▶ Mapping a parser result
- ▶ Sequencing parsers

Core concepts

- ▶ Parser α : input to `Option` result and remainder
- ▶ `mapParser`: result transformation
- ▶ `sequence`: ordered input consumption

Lean example

```
abbrev Parser ( $\alpha$  : Type) :=  
  List Char  $\rightarrow$  Option ( $\alpha \times$  List Char)  
  
def character (wanted : Char) : Parser Char  
| c :: rest =>  
  if c == wanted then some (c, rest)  
  else none  
| [] => none
```

Lean details

- ▶ Failure represented by none
- ▶ Unconsumed input preserved by mapping
- ▶ Remaining input passed to the next parser
- ▶ Mapping and sequencing as Functor and Applicative operations

Live coding

- ▶ `character`
- ▶ `mapParser`
- ▶ Parser for two characters

Check

Remainder after parsing ab from abc

Exercise

- ▶ Alternative character parser
- ▶ Parser trace